

Assignment 3 Report

MPI Branch-and-Bound for Maximum Profit Budgeted Clique

Arib Ashhar Shamoon

Entry Number: 2025MCS2013

COL7880: Parallel and Distributed Programming

April 14, 2026

1 Problem Statement

The task is to compute a maximum-profit clique under a global budget constraint. Each vertex v has an associated profit $p(v)$ and cost $c(v)$. The objective is to find a clique C such that:

$$\sum_{v \in C} c(v) \leq B$$

and the total profit

$$\sum_{v \in C} p(v)$$

is maximized.

The implementation follows the branch-and-bound framework required in the assignment and uses MPI for parallel execution. As clarified in the discussion thread, the report graph is plotted using results from a single node, with multiple MPI processes on that node.

2 Implementation Overview

The final solver is written in C++17 using MPI and uses a depth-first branch-and-bound search. The implementation keeps the assignment-mandated heuristic order:

1. Structural bound using greedy coloring.
2. Fractional knapsack bound.

The main implementation choices are listed below.

Graph representation. The graph is stored using flattened bitset adjacency rows. This reduces memory indirection and improves cache locality during repeated adjacency and color-class intersection checks.

Branching order. Vertices are initially ordered by profit-to-cost efficiency. This order is used to build the candidate lists and explore promising vertices early, which improves the quality of the incumbent solution and increases pruning.

Structural bound optimization. The coloring-based structural bound is implemented using contiguous memory for color classes. Candidate ordering for the color bound is produced using bucket placement by profit value, taking advantage of the bounded profit range in the input. The hot path for class-intersection checks is specialized for the small fixed number of machine words used by the report graph.

Initial incumbent generation. Before the full search begins, the code constructs several greedy feasible cliques using different seed strategies. This gives a strong initial lower bound and helps reduce the search tree size.

Parallel decomposition. For multiple MPI processes, work is split over a deeper frontier of the search tree. Each rank explores a subset of subproblems, and the best local solution is collected at the end on rank 0.

3 Benchmark Setup

Benchmarks were run on the provided report test graph `input_report.txt`. The benchmark script executes the program with:

$$\text{np} \in \{1, 2, 4, 8, 16\}$$

on a single node, as required for the report.

The recorded best profit for all runs was:

$$\text{profit} = 650$$

Two times are shown in the results:

- **Execution time (ms):** time reported by the program for the search section.
- **Wall-clock time (s):** time measured externally by the benchmark script.

4 Results

Table 1: Benchmark results on `input_report.txt`

np	Execution Time (ms)	Wall Time (s)	Speedup	Efficiency	Profit
1	3777	4.32	1.000	1.000	650
2	2070	2.60	1.825	0.912	650
4	1039	1.58	3.635	0.909	650
8	544	1.14	6.943	0.868	650
16	278	0.95	13.586	0.849	650

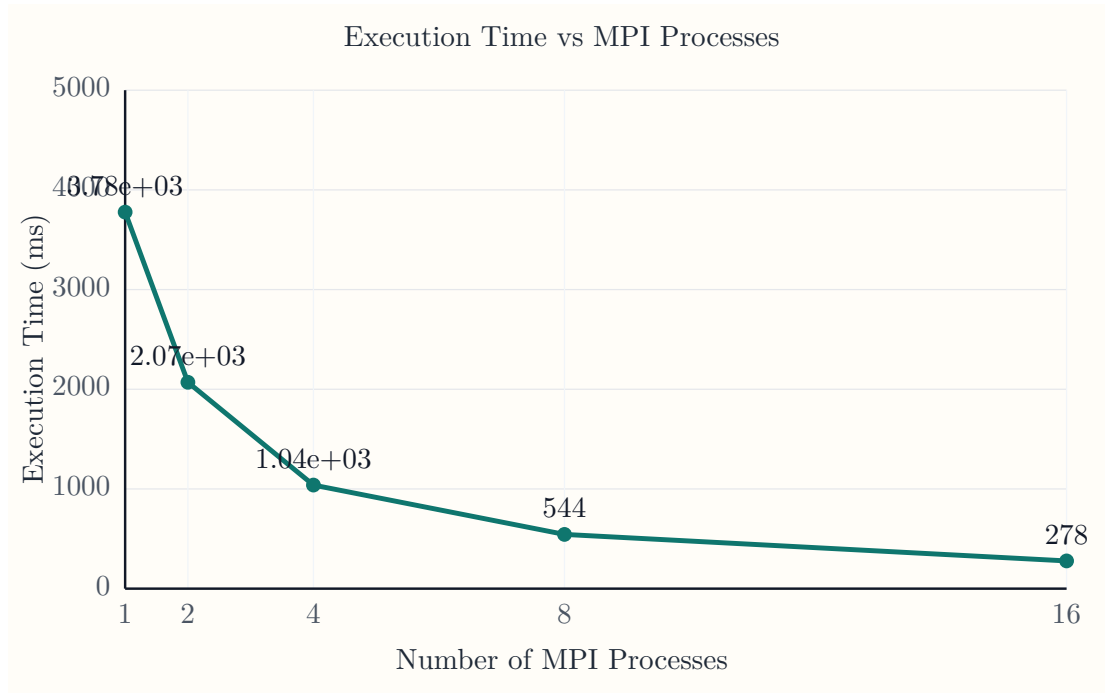


Figure 1: Execution time scaling with number of MPI processes

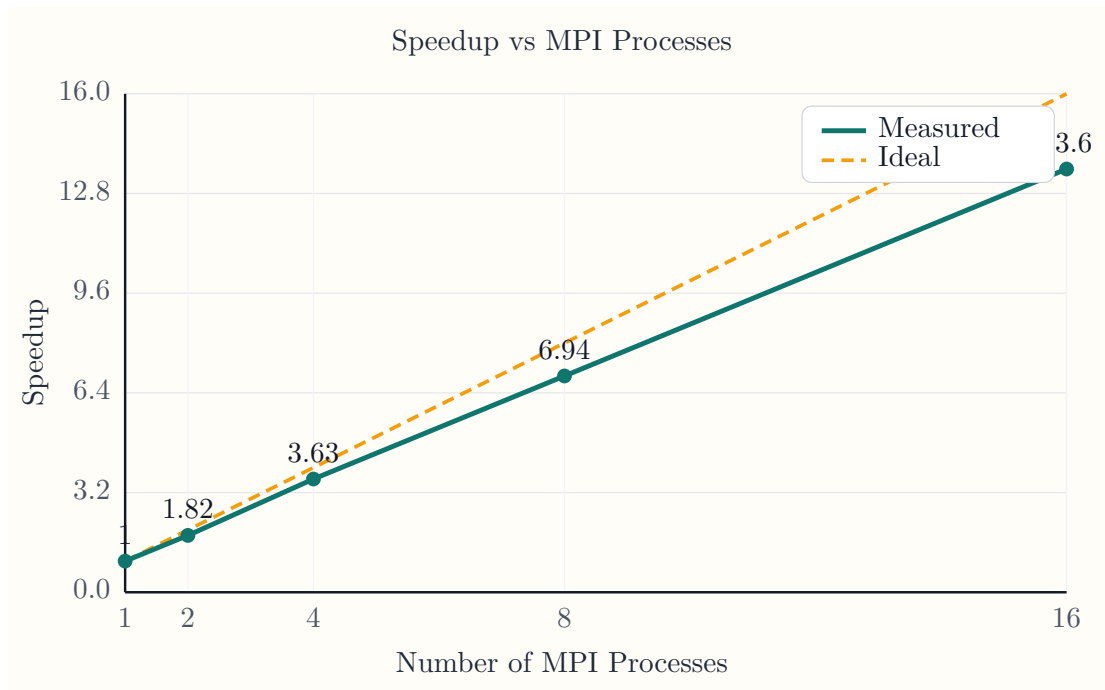


Figure 2: Speedup relative to the `np=1` run

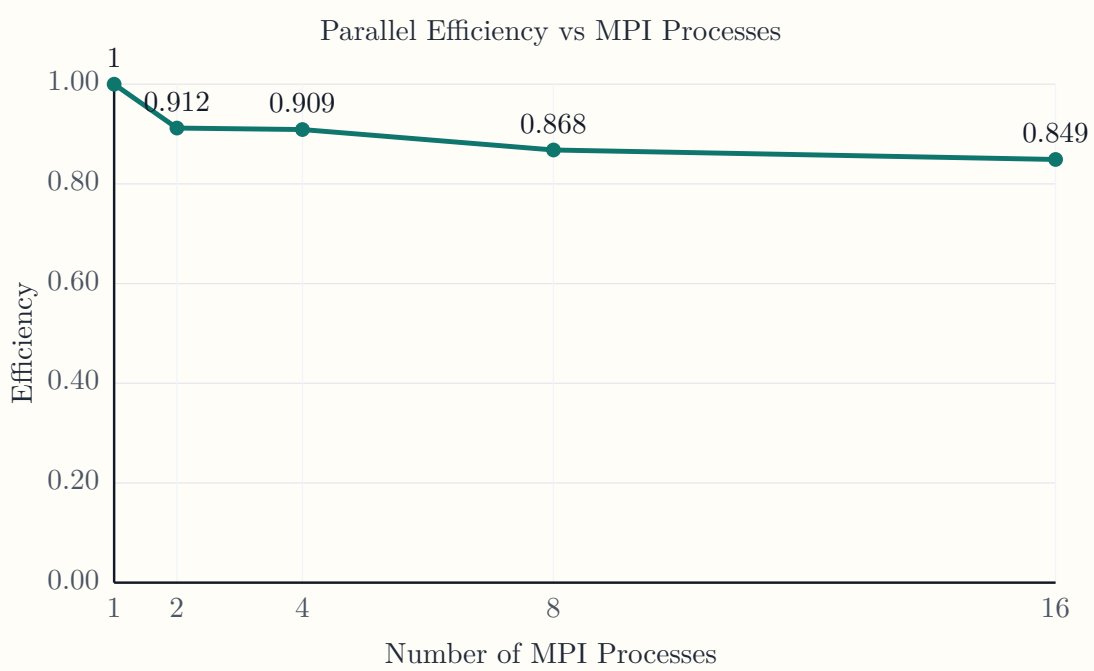


Figure 3: Parallel efficiency for different process counts

5 Discussion

The solver shows strong scaling on the report testcase. The internal execution time decreases from 3777 ms at `np=1` to 278 ms at `np=16`, which corresponds to a speedup of approximately $13.59\times$. The efficiency remains above 84% even at 16 MPI processes, which indicates that the parallel work decomposition is effective for this testcase.

The most important performance improvements in the final implementation came from reducing overhead inside the structural bound. In particular, flattening the adjacency and color-class memory layout and specializing the intersection checks for the report graph’s fixed bitset width significantly reduced the cost of the most frequently executed operations.

The wall-clock times remain slightly larger than the internal execution time due to MPI startup and process-management overhead. This effect is relatively more visible at higher process counts, but the overall scaling trend remains strong.

6 Conclusion

The final MPI implementation correctly computes the optimal profit 650 on the provided report testcase and achieves good parallel scalability on a single node. The final version preserves the required branch-and-bound structure while improving constant factors in the graph representation, color-bound computation, and search initialization.

Overall, the implementation demonstrates that careful optimization of the sequential branch-and-bound core is essential for obtaining strong speedup in the MPI version.